

# OVERVIEW OF C

## 1.1. INTRODUCTION

'C' seems a strange name for a programming language. But this strange sounding language is one of the most popular computer languages today. C was an offspring of the 'Basic Combined Programming Language' (BCPL) called B, developed in the 1960's at Cambridge University. B language was modified by Dennis Ritchie and was implemented at Bell Laboratories in 1972. The new language was named C. Since it was developed along with the UNIX operating system, it is strongly associated with UNIX. This operating system, which was also developed at Bell Laboratories, was coded almost entirely in C.

For many years, C was used mainly in academic environments, but eventually with the release of C compilers for commercial use and the increasing popularity of UNIX, it began to gain widespread support among computer professionals. Today, C is running under a number of operating systems including MS-DOS. Since MS-DOS is a dominant operating system for microcomputers, it is natural that C has begun to influence the microcomputer community at large.

This book describes the features of C that are generally supported by most compilers. It also includes all the essential features of ANSI C.

## 1.2 IMPORTANCE OF C

The increasing popularity of C is probably due to its many desirable qualities. It is a robust language whose rich set of built-in functions and operators can be used to write any complex program. The C compiler combines the capabilities of an assembly language with the features of a high-level language and therefore it is well suited for writing both system software and business packages. In fact, many of the C compilers available in the market are written in C.

Programs written in C are efficient and fast. This is due to its variety of data types and powerful operators. It is many times faster than BASIC. For example, a program to increment a variable from 0 to 15000 takes about one second in C while it takes more than 50 seconds in an interpreter BASIC.

There are only 32 keywords and its strength lies in its built-in functions. Several standard functions are available which can be used for developing programs.

C is highly portable. This means that C programs written for one computer can be run on another with little or no modification. Portability is important if we plan to use a new computer with a different operating system.

## 2 PROGRAMMING IN ANSI C

C language is well suited for structured programming, thus requiring the user to think of a problem in terms of function modules or blocks. A proper collection of these modules would make a complete program. This modular structure makes program debugging, testing and maintenance easier.

Another important feature of C is its ability to extend itself. A C program is basically a collection of functions that are supported by the C library. We can continuously add our own functions to the C library. With the availability of a large number of functions, the programming task becomes simple.

### 1.3 SAMPLE C PROGRAMS

Before discussing any specific features of C, we shall look at some sample C programs and analyse and understand how they work.

#### Sample Program 1: Printing a Message

Consider a very simple program given in Fig. 1.1. This program when executed, will produce the following output:

I see, I remember

```
main( )
{
/* .....printing begins..... */
    printf("I see, I remember");
/* .....printing ends ..... */
}
```

**Fig. 1.1** A program to print one line of text

Let us have a close look at the program. The first line informs the system that the name of the program is **main** and the execution begins at this line. The **main( )** is a special function used by the C system to tell the computer where the program starts. Every program must have *exactly one* **main** function. If we use more than one **main** function, the compiler cannot tell which one marks the beginning of the program.

The empty pair of parentheses immediately following **main** indicates that the function main has no arguments (or parameters). The concept of arguments will be discussed in detail later when we discuss functions (in Chapter 9).

The opening brace '{' in the second line marks the beginning of the function **main** and the closing brace '}' in the last line indicates the end of the function. In this case, the closing brace also marks the end of the program. All the statements between these two braces form the function body. The function body contains a set of instructions to perform the given task.

In this case, the function body contains three statements, out of which only the **printf** line is an executable statement. The lines beginning with **/\*** and ending with **\*/** are known as *comment lines*. These are used in a program to enhance its readability and understanding. Comment lines are not executable statements and therefore anything between **/\*** and **\*/** is ignored by the



compiler. In general, a comment can be inserted anywhere blank spaces can occur—at the beginning, middle, or end of a line, but never in the middle of a word.

Because comments do not affect the execution speed and the size of a program, we should use them liberally in our programs. They help the programmers and other users in understanding the various functions and operations of a program and serve as an aid to debugging and testing. We shall see the use of comment lines more in the examples that follow.

Let us now look at the `printf()` function, the only executable statement of the program.

```
printf("I see, I remember");
```

`printf` is a predefined, standard C function for printing output. *Predefined* means that it is a function that has already been written and compiled, and linked together with our program at the time of linking. The concepts of compilation and linking are explained later in this chapter (Section 1.6). The `printf` function causes everything between the starting and the ending quotation marks to be printed out. In this case, the output will be:

```
I see, I remember
```

Note that the print line ends with a semicolon. *Every statement in C should end with a semicolon (;) mark.*

Suppose we want to print the above quotation in two lines as

```
I see,  
I remember !
```

This can be achieved by adding *another* `printf` function as shown below:

```
printf("I see, \n");  
printf("I remember !");
```

The information contained between the parentheses is called the *argument* of the function. The argument of the first `printf` function is "I see,\n" and the second is "I remember !". These arguments are simply strings of characters to be printed out.

Notice that the argument of the first `printf` contains a combination of two characters \ and n at the end of the string. This combination is collectively called the *newline* character. A newline character instructs the computer to go to the next (new) line. It is similar in concept to the carriage return key on a typewriter. After printing the character comma (,) the presence of the newline character \n causes the string "I remember" to be printed on the next line. No space is allowed between \ and n.

If we omit the newline character from the first `printf` statement, then the output will again be a single line as shown below.

```
I see,I remember !
```

This is similar to the output of the program in Fig. 1.1. However, note that there is no space between , and I.

It is also possible to produce two or more lines of output by one `printf` statement with the use of newline characters at appropriate places. For example, the statement

```
printf("I see,\n I remember !");
```

will output

#### 4 PROGRAMMING IN ANSI C

```
I see,  
I remember !
```

while the statement

```
printf("\n.see,\n.....\n.....remember !");
```

will print out

```
I  
. see,  
.....  
..... remember !
```

Before we proceed to discuss further examples, we must note one important point. C does make a distinction between *uppercase* and *lowercase* letters. For example, **printf** and **PRINTF** are not the same. In C, everything is written in lowercase letters. However, uppercase letters are used for symbolic names representing constants. We may also use uppercase letters in output strings like "I SEE" and "I REMEMBER".

The above example that printed **I see, I remember** is one of the simplest programs. Figure 1.2 highlights the general format of such simple programs. All C programs need a **main** function.

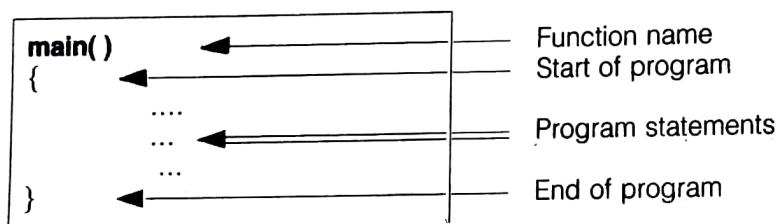


Fig. 1.2 Format of simple C programs

#### Sample Program 2: Adding Two Numbers

Consider another program which performs addition on two numbers and displays the result. The complete program is shown in Fig. 1.3.

```
/* Program ADDITION                                line-1 */  
/* Written by EBG                                  line-2 */  
main( )                                           /* line-3 */  
{                                                 /* line-4 */  
    int  number;                                  /* line-5 */  
    float amount;                                /* line-6 */  
                                                /* line-7 */  
    number = 100;                                /* line-8 */  
                                                /* line-9 */  
    amount = 30.75 + 75.35;                       /* line-10 */  
    printf("%d\n", number);                       /* line-11 */  
    printf("%5.2f", amount);                      /* line-12 */  
}                                                  /* line-13 */
```

Fig. 1.3 Program to add two numbers

This program when executed will produce the following output:

```
100
106.10
```

The first two lines of the program are comment lines. It is a good practice to use comment lines in the beginning to give information such as name of the program, author, date, etc. Comment characters are also used in other lines to indicate line numbers.

The words **number** and **amount** are *variable names* that are used to store numeric data. The numeric data may be either in *integer* form or in *real* form. In C, *all variables should be declared*, to tell the compiler what the variable names are and what *type of data* they hold. The variables must be declared before they are used. In lines 5 and 6, the declarations

```
int number;
float amount;
```

tell the compiler that **number** is an integer (int is the abbreviation for integer) and **amount** is a floating (**float**) point number. Declaration statements must appear at the beginning of the function as shown in Fig. 1.3. All declaration statements end with a semicolon. C supports many other data types and they are discussed in detail in Chapter 2.

The words such as **int** and **float** are called the *keywords* and cannot be used as variable names. A list of keywords is given in Chapter 2.

Data is stored in a variable by *assigning* a data value to it. This is done in lines 8 and 10. In line-8, an integer value 100 is assigned to the integer variable **number** and in line-10, the result of addition of two real numbers 30.75 and 75.35 is assigned to the floating point variable **amount**. The statements

```
number = 100;
amount = 30.75 + 75.35;
```

are called the *assignment* statements. Every assignment statement must have a semicolon at the end.

The next statement is an output statement that prints the value of **number**. The print statement

```
printf("%d\n", number);
```

contains two arguments. The first argument "**%d**" tells the compiler that the value of the second argument **number** should be printed as a *decimal integer*. Note that these arguments are separated by a comma. The newline character **\n** causes the next output to appear on a new line.

The last statement of the program

```
printf("%5.2f", amount);
```

prints out the value of **amount** in floating point format. The format specification **%5.2f** tells the compiler that the output must be in floating point, with five places in all and two places to the right of the decimal point.

### Sample Program 3 : Interest Calculation

The program in Fig. 1.4 calculates the value of money at the end of each year of investment,



assuming an interest rate of 11 percent and prints the year and the corresponding amount, in two columns. The output is shown in Fig. 1.5 for a period of 10 years with an initial investment of 5000.00. The program uses the following formula:

$$\text{Value at end of year} = \text{Value at start of year} (1 + \text{interest rate})$$

In the program, the variable **value** represents the value of money at the end of the year while **amount** represents the value of money at the start of the year. The statement

**amount = value;**

makes the value at the end of the *current* year as the value at the start of the *next* year.

```

/* .....INVESTMENT PROBLEM..... */
#define PERIOD    10
#define PRINCIPAL 5000.00
/* .....MAIN PROGRAM BEGINS..... */
main()
{
/* .....DECLARATION STATEMENTS..... */
    int year;
    float amount, value, inrate;

/* .....ASSIGNMENT STATEMENTS..... */
    amount = PRINCIPAL;
    inrate = 0.11;
    year = 0;

/* .....COMPUTATION STATEMENTS..... */
/* .....COMPUTATION USING while LOOP..... */
    while(year <= PERIOD)
    {
        printf("%2d      %8.2fn", year, amount);
        value = amount + inrate * amount;
        year = year + 1;
        amount = value;
    }

/* .....while LOOP ENDS..... */
}
/* .....PROGRAM ENDS..... */

```

**Fig. 1.4** Program for investment problem

Let us consider the new features introduced in this program. The second and third lines begin with **#define** instructions. A **#define** instruction defines value to a *symbolic* constant for use in the program. Whenever a symbolic name is encountered, the compiler substitutes the value associated with the name automatically. To change the value, we have to simply change the definition. In this example, we have defined two symbolic constants **PERIOD** and **PRINCIPAL** and assigned values 10 and 5000.00 respectively. These values remain constant throughout the execution of the program.

|    |          |
|----|----------|
| 0  | 5000.00  |
| 1  | 5550.00  |
| 2  | 6160.50  |
| 3  | 6838.15  |
| 4  | 7590.35  |
| 5  | 8425.29  |
| 6  | 9352.07  |
| 7  | 10380.80 |
| 8  | 11522.69 |
| 9  | 12790.19 |
| 10 | 14197.11 |

**Fig. 1.5** Output of the investment program

A **#define** is a compiler directive and not a statement. Therefore **#define** lines should not end with a semicolon. Symbolic constants are generally written in uppercase so that they are easily distinguished from lowercase variable names. **#define** instructions are usually placed at the beginning, before the **main( )** function. Symbolic constants are not declared in declaration section.

We must note that the defined constants are not variables. We may not change their values within the program by using an assignment statement. For example, the statement

**PRINCIPAL = 10000.00;**

is illegal.

The declaration section declares **year** as integer and **amount**, **value**, and **inrate** as floating point numbers. Note that all the floating point variables are declared in one statement. They can also be declared as

```
float amount;
float value;
float inrate;
```

When two or more variables are declared in one statement, they are separated by a comma.

All computations and printing are accomplished in a **while** loop. **While** is a mechanism for evaluating repeatedly a statement or a group of statements. In this case as long as the value of **year** is less than or equal to the value of **PERIOD**, the four statements that follow **while** are executed. Note that these four statements are grouped by braces. We exit the loop when **year** becomes greater than **PERIOD**.

**C** supports the basic four arithmetic operators (**-**, **+**, **\***, **/**) along with several others. They are discussed in Chapter 3.

### Sample Program 4: Use of Subroutines

So far, we have used only **printf** function that has been provided for us by the C system. The program shown in Fig. 1.6 uses a user-defined function. A function defined by the user is equivalent to a subroutine in FORTRAN or subprogram in BASIC.

```

/* ..... PROGRAM USING FUNCTION ..... */
/* ..... MAIN PROGRAM BEGINS ..... */
main( )
{
    int  a, b, c;
    a = 5;
    b = 10;
    c = mul(a,b);
    printf("Multiplication of %d and %d is %d",
           a,b,c);
}
/* ..... MAIN PROGRAM ENDS
   MUL( ) FUNCTION STARTS ..... */

mul (x,y)
int p,x,y; /* ..... ARGUMENT DECLARATIONS ..... */
{
    p = x*y;
    return(p);
}
/* ..... MUL( ) FUNCTION ENDS..... */

```

**Fig. 1.6** A program using a user-defined function.

Figure 1.6 presents a very simple program that uses a **mul( )** function. The program will print the following output:

Multiplication of 5 and 10 is 50

The **mul(x,y)** multiplies the values of **x** and **y** and the result is returned to the **main( )** function when it is called in the statement

**c = mul(a,b);**

The **mul( )** has two *arguments* **x** and **y** that are declared as integers. The values of **a** and **b** are passed on to **x** and **y** respectively when the function **mul( )** is called. User-defined functions are considered in detail in Chapter 9.

### Sample Program 5: Use of Math Functions

We often use standard mathematical functions such as **cos**, **sin**, **exp**, etc. We shall see now the use of a mathematical function in a program. The standard mathematical functions are defined and kept as a part of C **math library**. If we want to use any of these mathematical functions, we must add an **#include** instruction in the program. Like **#define**, it is also a compiler directive that instructs the compiler to link the specified mathematical functions from the library. The instruction is of the form

**#include <math.h>**

**math.h** is the filename containing the required function. Figure 1.7 illustrates the use of cosine



function. The program calculates cosine values for angles 0,10,20.....,180 and prints out the results with headings.

Another **#include** instruction that is often required is

**#include <stdio.h>**

**stdio.h** refers to the *standard I/O* header file containing standard input and output functions.

```

/*****
/*      PROGRAM USING COSINE FUNCTION      */
/*****
#include <math.h>
#define PI      3.1416
#define MAX     180
main()
{
    int  angle;
    float x,y;
    angle = 0;
    printf("      Angle      Cos(angle)\n\n")
    while(angle <= MAX)
    {
        x = (PI/MAX)*angle;
        y = cos(x);
        printf("%15d %13.4f\n", angle, y);
        angle = angle + 10;
    }
}

```

*Output*

| Angle | Cos(angle) |
|-------|------------|
| 0     | 1.0000     |
| 10    | 0.9848     |
| 20    | 0.9397     |
| 30    | 0.8660     |
| 40    | 0.7660     |
| 50    | 0.6428     |
| 60    | 0.5000     |
| 70    | 0.3420     |
| 80    | 0.1736     |
| 90    | -0.0000    |
| 100   | -0.1737    |
| 110   | -0.3420    |
| 120   | -0.5000    |
| 130   | -0.6428    |
| 140   | -0.7660    |
| 150   | -0.8660    |
| 160   | -0.9397    |
| 170   | -0.9848    |
| 180   | -1.0000    |

**Fig. 1.7** Program using a math function

1.4 BASIC STRUCTURE OF C PROGRAMS

The examples discussed so far illustrate that a C program can be viewed as a group of building blocks called *functions*. A function is a subroutine that may include one or more *statements* designed to perform a *specific task*. To write a C program, we first create functions and then put them together. A C program may contain one or more sections shown in Fig. 1.8.

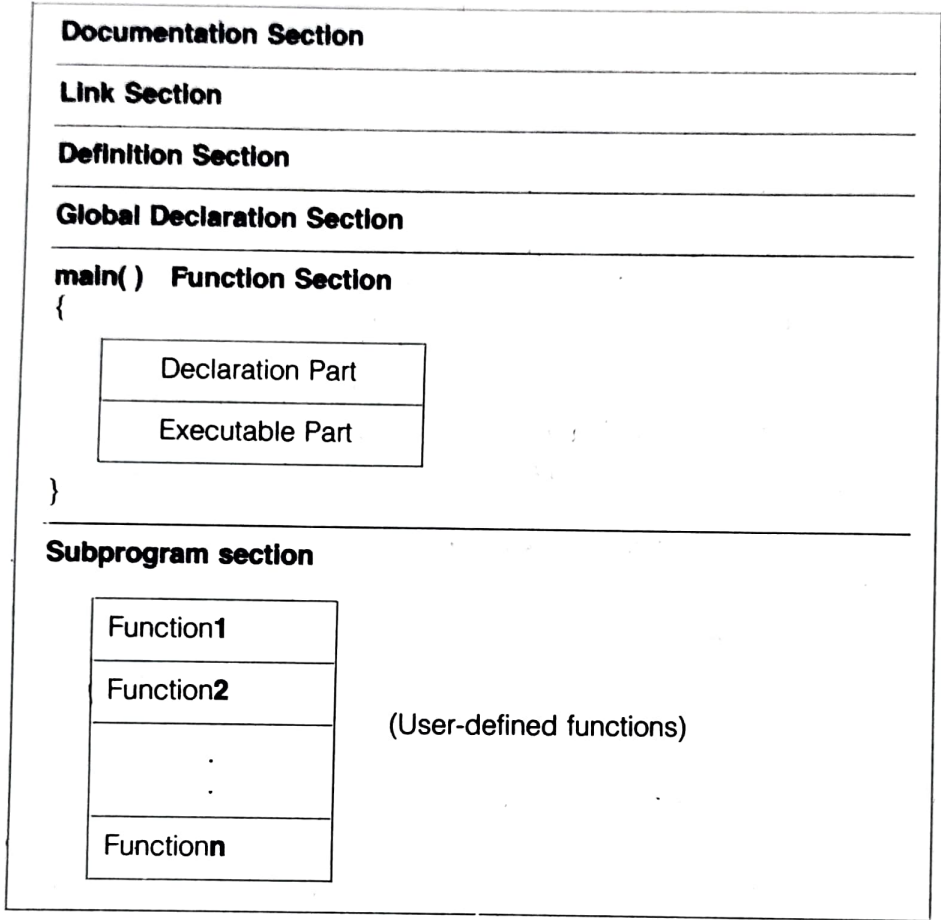


Fig. 1.8 An overview of a C program

The documentation section consists of a set of comment lines giving the name of the program, the author and other details which the programmer would like to use later. The link section provides instructions to the compiler to link functions from the system library. The definition section defines all symbolic constants.

There are some variables that are used in more than one function. Such variables are called *global variables* and are declared in the global declaration section that is outside of all the functions.

Every C program must have one **main( )** function section. This section contains two parts, declaration part and executable part. The declaration part declares all the variables used in the executable part. There is at least one statement in the executable part. These two parts must appear between the opening and the closing braces. The program execution begins at the opening brace and ends at the closing brace. The closing brace of the main function section is

the logical end of the program. All statements in the declaration and executable parts end with a semicolon.

The subprogram section contains all the user-defined functions that are called in the **main** function. User-defined functions are generally placed immediately after the **main** function, although they may appear in any order.

All sections, except the **main** function section may be absent when they are not required.

## 1.5 PROGRAMMING STYLE

Unlike some other programming languages (COBOL, FORTRAN, etc.), C is a *free-form* language. That is, the C compiler does not care, where on the line we begin typing. While this may be a license for bad programming, we should try to use this fact to our advantage in developing readable programs. Although several alternative styles are possible, we should select one style and use it with total consistency.

First of all, we must develop the habit of writing programs in lowercase letters. C program statements are written in lowercase letters. Uppercase letters are used only for symbolic constants.

Braces group program statements together and mark the beginning and the end of functions. A proper indentation of braces and statements would make a program easier to read and debug. Note how the braces are aligned and the statements are indented in the program of Fig. 1.4.

Since C is a free-form language, we can group statements together on one line. The statements

```
a = b;
x = y+1;
z = a+x;
```

can be written in one line as

```
a = b; x = y+1; z = a+x;
```

In this book, each statement is written on a separate line.

The generous use of comments inside a program cannot be overemphasized. Judiciously inserted comments not only increase the readability but also help to understand the program logic. This is very important for debugging and testing the program.

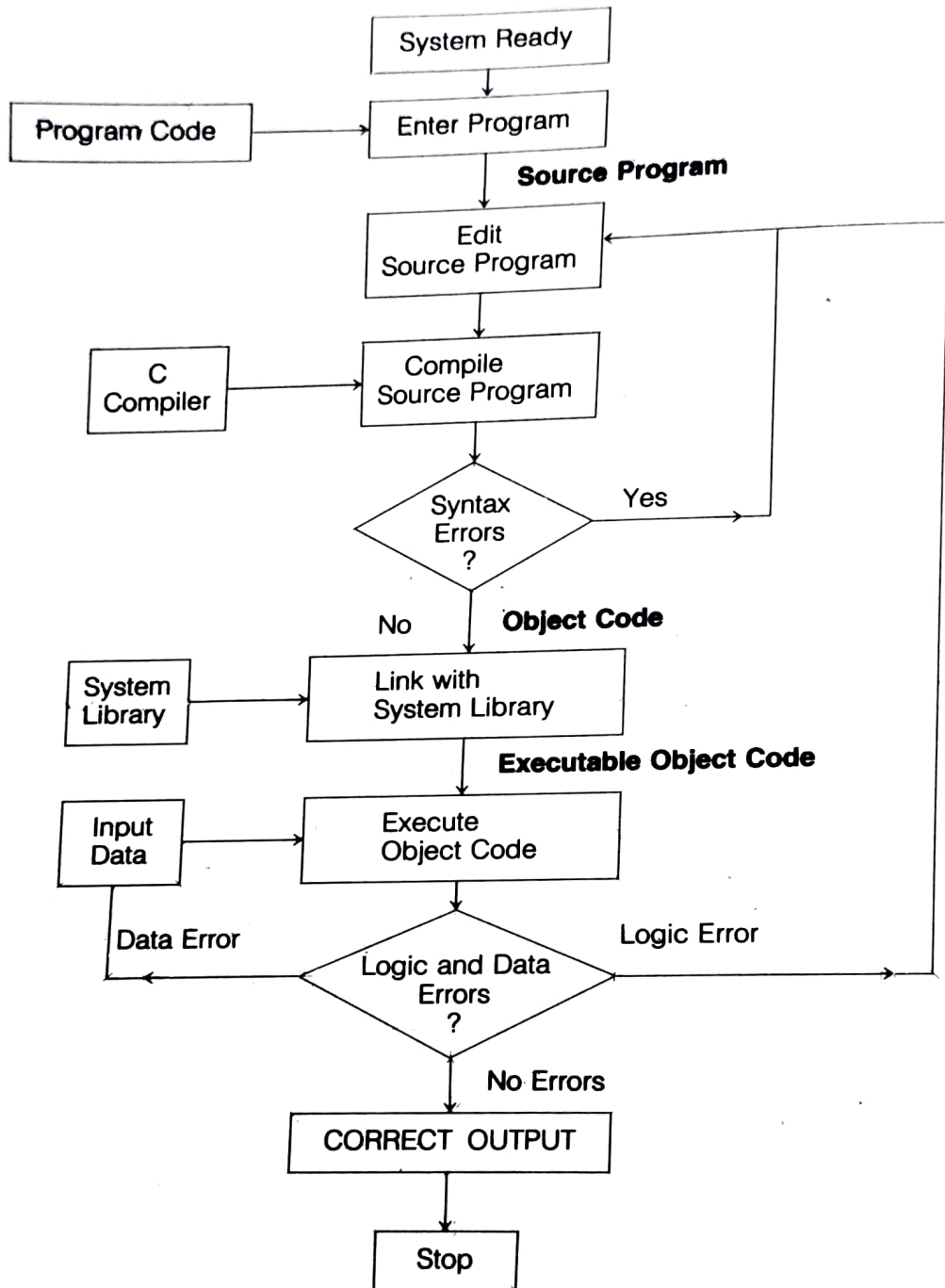
## 1.6 EXECUTING A 'C' PROGRAM

Executing a program written in C involves a series of steps. These are:

1. Creating the program.
2. Compiling the program.
3. Linking the program with functions that are needed from the C library.
4. Executing the program.

Figure 1.9 illustrates the process of creating, compiling and executing a C program. Although these steps remain the same irrespective of the *operating system*, system commands for implementing the steps and conventions for naming *files* may differ on different systems.





**Fig. 1.9** Process of compiling and running a C program

An operating system is a program that controls the entire operation of a computer system. All input/output operations are channelled through the operating system. The operating system, which is an interface between the hardware and the user, handles the execution of user programs.

The two most popular operating systems today are UNIX (for minicomputers) and MS-DOS (for microcomputers). We shall discuss briefly the procedure to be followed in executing C programs under both these operating systems.

## UNIX System

### Creating the program

Once we load the UNIX Operating system into the memory, the computer is ready to receive the program. The program must be entered into a *file*. The file name can consist of letters, digits and special characters, followed by a *dot* and a letter *c*. Examples of valid file names are:

*hello.c*

*program.c*

*ebg1.c*

The file is created with the help of a text *editor*, either **ed** or **vi**. The command for calling the editor and creating the file is

**ed filename**

If the file existed before, it is loaded. If it does not yet exist, the file has to be created so that it is ready to receive the new program. Any corrections in the program are done under the editor. (The name of your system's editor may be different. Check your system manual.)

When the editing is over, the file is saved on disk. It can then be referenced any time later by its file name. The program that is entered into the file is known as the *source program*, since it represents the original form of the program.

### Compiling and Linking

Let us assume that the program has been created in a file named *ebg1.c*. Now the program is ready for compilation. The compilation command to achieve this task under UNIX is

**cc ebg1.c**

The source program instructions are now translated into a form that is suitable for execution by the computer. The translation is done after examining each instruction for its correctness. If everything is alright, the compilation proceeds silently and the translated program is stored on another file with the name *ebg1.o*. This program is known as *object code*.

Linking is the process of putting together other program files and functions that are required by the program. For example, if the program is using **exp()** function, then the object code of this function should be brought from the **math library** of the system and linked to the main program. Under UNIX, the linking is automatically done (if no errors are detected) when the **cc** command is used.

## 14 PROGRAMMING IN ANSI C

If mistakes in the *syntax* and *semantics* of the language are discovered, they are listed out and the compilation process ends right there. The errors should be corrected in the source program with the help of the editor and the compilation is done again.

The compiled and linked program is called the *executable object code* and is stored automatically in another file named **a.out**.

Note that some systems use different compilation command for linking mathematical functions.

**cc filename -lm**

is the command under UNIPLUS SYSTEM V operating system.

### Executing the Program

Execution is a simple task. The command

**a.out**

would load the executable object code into the computer memory and execute the instructions. During execution, the program may request for some data to be entered through the keyboard. Sometimes the program does not produce the desired results. Perhaps, something is wrong with the program *logic* or *data*. Then it would be necessary to correct the source program or the data. In case the source program is modified, the entire process of compiling, linking and executing the program should be repeated.

### Creating Your Own Executable File

Note that the linker always assigns the same name **a.out**. When we compile another program, this file will be over written by the executable object code of the new program. If we want to prevent from happening, we should rename the file immediately by using the command

**mv a.out name**

We may also achieve this by specifying an option in the **cc** command as follows:

**cc -o name source-file**

This will store the executable object code in the file name and prevent the old file **a.out** from being destroyed.

### Multiple Source Files

To compile and link multiple source program files, we must append all the file names to the **cc** command.

**cc filename-1.c ..... filename-n.c**

These files will be separately compiled into object files called

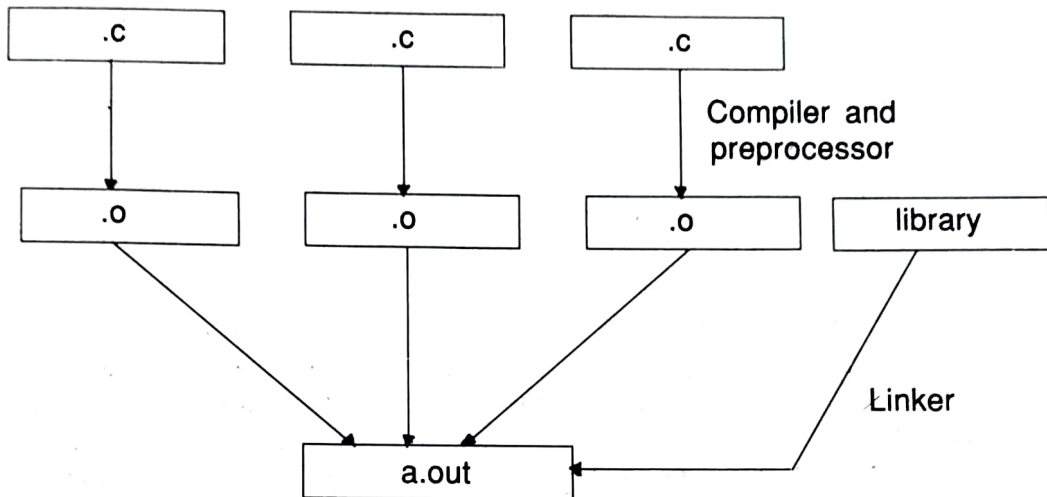
**filename-1.o**



and then linked to produce an executable program file **a.out** as shown in Fig. 1.10.

It is also possible to compile each file separately and link them later. For example, the commands

```
cc -c mod1.c
cc -c mod2.c
```



**Fig. 1.10** *Compilation of multiple files*

will compile the source files *mod1.c* and *mod2.c* into object files *mod1.o* and *mod2.o*. They can be linked together by the command

```
cc mod1.o mod2.o
```

We may also combine the source files and object files as follows:

```
cc mod1.c mod2.o
```

Only *mod1.c* is compiled and then linked with the object file *mod2.o*. This approach is useful when one of the multiple source files need to be changed and recompiled or an already existing object file is to be used along with the program to be compiled.

## MS-DOS System

The program can be created using any word processing software in non-document mode. The file name should end with the characters ".c", like **program.c**, **pay.c**, etc. Then the command

```
MSC pay.c
```

under MS-DOS operating system would load the program stored in the file **pay.c** and generate the **object code**. This code is stored in another file under the name **pay.obj**. In case any language errors are found, the compilation is not completed. The program should then be corrected and compiled again.

The linking is done by the command

**LINK pay.obj**

which generates the executable code with the filename **pay.exe**. Now the command

**pay**

would execute the program and give the results.

## 1.7 POINTS TO REMEMBER

1. Every C program requires a **main( )** function (Use of more than one **main( )** is illegal.). The place **main** is where the program execution begins.
2. The execution of a function begins at the opening brace of the function and ends at the corresponding closing brace.
3. C programs are written in lowercase letters. However, uppercase letters are used for symbolic names and output strings.
4. All the words in a program line must be separated from each other by at least one space, or a tab, or a punctuation mark.
5. Every program statement in a C program must end with a semicolon.
6. All variables must be declared for their types before they are used in the program.
7. We must make sure to include *header files* using **#include** directive when the program refers to special names and functions that it does not define.
8. Compiler directives such as **define** and **include** are special instructions to the compiler to help it compile a program. They do not end with a semicolon.
9. The sign **#** of compiler directives must appear in the first column of the line.
10. When braces are used to group statements, make sure that the opening brace has a corresponding closing brace.
11. C is a free-form language and therefore a proper form of indentation of various sections would improve legibility of the program.
12. A comment can be inserted almost anywhere a space can appear. Use of appropriate comments in proper places increases readability and understandability of the program and helps users in debugging and testing. Remember to match the symbols **/\*** and **\*/** appropriately.

## REVIEW QUESTIONS AND EXERCISES

- 1.1 If you have access to a computer system with the C compiler, find out the details of the operating system and special features of the compiler.
- 1.2 Compile and run all the programs discussed in this chapter. Compare the outputs with those presented with the programs.
- 1.3 Remove the semicolon at the end of the **printf** statement in the program of Fig. 1.1 and try to execute it. What is the output of the program?
- 1.4 In the Sample Program 2, delete line-5 and execute the program. How helpful is the error message?
- 1.5 Modify the Sample Program 3 to print the following output:

| Year | Amount   |
|------|----------|
| 1    | 5550.00  |
| 2    | 6160.00  |
| .    | .        |
| .    | .        |
| 10   | 14197.11 |

Note that the modified output contains only 10 values (for Year = 1,...,10).

1.6 Identify syntax errors in the following program:

```

Include < math.h>
main{ }
(
    FLOAT X;
    /** ASSIGNMENT SECTION**
    X = 2.5,
    Y = exp (x);
    /** PRINTING SECTION**/
    print(x, y)
)

```

1.7 Write a program that will print your mailing address in the following form:

First line: Name  
 Second line: Door No, Street  
 Third line: City, Pin code

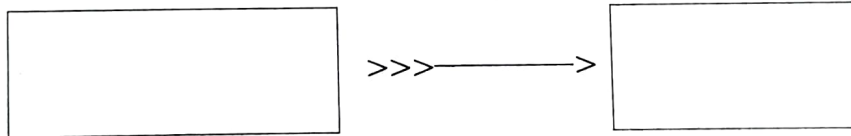
1.8 Write a program to output the following multiplication table:

```

5 × 1  = 5
5 × 2  = 10
5 × 3  = 15
.
.
.
5 × 20 = 100

```

1.9 Write a program that will print the following figure:



1.10 State whether the following statements are TRUE or FALSE:

- Every line in a C program should end with a semicolon.
- In C language, lowercase letters are significant.
- Every C program ends with an END word.
- main( ) is where the program begins its execution.
- A line in a C program may have more than one statement.
- A printf( ) statement can generate *only one* line of output.
- The closing brace of the main( ) in a program is the logical end of the program.
- Syntax errors will be detected by the compiler.